Coding & Solution

Date	01 July 2026
Team ID	
Project Name	Smart Lender Loan Eligibility Prediction System
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/BandlamudiVengaManikanta/AIML-Smart_Lender
Programming Language(s)	Python (for ML + backend), HTML/CSS/JavaScript (for UI)
Framework(s) Used	Flask (backend web framework), Bootstrap (frontend styling)

Key Features Implemented	- User registration & login (JWT authentication) - Loan application form (income, loan amount, credit history, property area) - ML-based prediction using XGBoost - Real-time decision-making dashboard for credit officers - Reporting module with risk scores and approval rates
Pending / Incomplete Features	- Facebook login integration - Advanced audit logging with compliance dashboards - Full CI/CD pipeline automation
Setup / Run Instructions	1. Clone repository: git clone <repo-url> 2. Install dependencies: pip install -r requirements.txt 3. Run Flask app: python app.py 4. Access via browser: http://localhost:5000

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Partial (ML scripts well-commented, Flask routes need more docstrings)
4	Error handling is implemented for critical operations	Partial (basic try-except blocks, logging can be improved)
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes (GitHub)

Additional Notes / Comments:

- The ML pipeline is modularized (preprocessing, training, prediction).
- XGBoost chosen as final model due to highest accuracy.
- Flask routes are functional but need more inline documentation.
- Error handling and logging should be enhanced for production readiness.